Connected Component Analysis for Multi-Vehicle Detection in Shipment Tracking Systems

Bachelors of Technology

in

Information Technology
Bachelor's of Technology Project Report

by

Aditya Kumar Singh (2022IMG-005)

Under the supervision of

Prof. Kiran Kumar Pattanaik

विश्वजीवनामृतं ज्ञानम्

ABV INDIAN INSTITUTE OF INFORMATION TECHNOLOGY AND MANAGEMENT

GWALIOR-474 015

CANDIDATE DECLARATION

I hereby certify that the work, which is being presented in the report, entitled **Connected Component Analysis for Multi-Vehicle Detection in Shipment Tracking Systems**, in fulfillment of the requirement for the award of the Degree of **Bachelor of Technology (B.Tech)** and submitted to the institution is an authentic record of my work carried out during the period *January 2026* to *March 2026* under the supervision of **Prof. Kiran Kumar Pattanaik**. I have also cited the reference to the text(s)/figure(s)/table(s) from where they have been taken.

Date: Signature of the Candidate

This is to confirm that the candidate's above statement is true to the best of my knowledge.

Date: Signature of the Project Supervisor

ABSTRACT

Shipment tracking systems depend on GPS position data to provide real-time visibility, accurate ETAs, and reliable operational insights. In production environments, this data is often affected by device errors, data integration issues, and system inconsistencies. One critical failure case occurs when a single shipment is incorrectly associated with multiple tracking vehicles, leading to physically impossible movement patterns such as long-distance jumps, oscillating locations, or simultaneous positions in different places. These errors severely impact customer trust, billing accuracy, and operational decision-making.

This report proposes a graph-theoretic framework to detect and validate such multi-vehicle tracking anomalies. Each GPS position is modeled as a node in a directed graph, and edges are created only when movement between two positions is physically feasible within realistic speed and time constraints. The resulting Position Reachability Graph allows shipment trajectories to be analyzed through connectivity rather than ad-hoc rules.

Connected components in the graph represent sets of positions that could belong to a single vehicle. The presence of multiple large, disconnected components indicates concurrent tracking by multiple vehicles. To account for real-world GPS noise, the framework incorporates jitter tolerance, ensuring that minor measurement errors do not cause incorrect fragmentation of valid trajectories.

Legitimate vehicle handovers, such as inter-modal transfers at ports or hubs, are handled using a secondary clustering and merging process based on spatial and temporal proximity. This prevents valid handovers from being misclassified as errors.

The proposed approach is mathematically rigorous, computationally efficient, and suitable for large-scale deployment. By framing shipment validation as a graph connectivity problem, the system provides a robust, explainable, and reliable solution for improving data quality in shipment tracking platforms.

Keywords: Shipment Tracking, Multi-Vehicle Detection, Connected Components, Position Reachability Graph, GPS Validation, Graph Theory

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Prof. Kiran Kumar Pattanaik for his invaluable guidance, encouragement, and support throughout the course of this project. His insightful feedback, expert advice, and constructive criticism have been instrumental in shaping my understanding of the subject and refining my approach. His constant motivation and belief in my abilities have inspired me to explore new ideas and push my boundaries.

I am also thankful to my institution, ABV-IIITM Gwalior, for providing me with the necessary resources and a conducive learning environment to conduct this research. Additionally, I extend my appreciation to my peers and colleagues for their insightful discussions, constructive feedback, and unwavering support, which have played a significant role in the development of this work.

Finally, I am grateful to my family and friends for their constant encouragement and support, which has helped me stay motivated and focused throughout this journey.

Aditya Kumar Singh

TABLE OF CONTENTS

ABSTRACT ii

ACKNOWLEDGEMENT iii

LIST OF TABLES v

LIST OF FIGURES vi

1 PROBLEM STATEMENT 1

1.1 Introduction . . . 1
1.2 Problem: Shipment Tracked by Multiple Vehicles . . . 2
1.2.1 Problem Description . . . 2
1.2.2 Observable Symptoms . . . 3
1.2.3 Legitimate Exception: Vehicle Handovers . . . 4
1.2.4 The Validation Challenge . . . 4
1.2.5 Business Impact . . . 4
1.3 System Overview . . . 7

2 GRAPH THEORY BACKGROUND 9

2.1 Graph Theory Fundamentals . . . 9
2.1.1 Basic Definitions . . . 9
2.1.2 Paths and Connectivity . . . 9
2.1.3 Special Graph Types . . . 10
2.1.4 Graph Representations . . . 10
2.2 Spatial Data Concepts . . . 11
2.2.1 Geographic Coordinate Systems . . . 11
2.2.2 Distance Calculations . . . 11
2.2.3 Speed Constraints and Reachability . . . 12
2.3 Notation Index . . . 12
2.4 Graph Construction Overview . . . 12

3 GRAPH FORMULATION 14

3.1 Introduction . . . 14
3.2 Position Reachability Graph . . . 14
3.2.1 Intuition . . . 14
3.2.2 Formal Definitions . . . 14
3.2.3 Graph Properties . . . 15
3.2.4 Extended Reachability with GPS Jitter Tolerance . . . 15
3.2.5 Handover Graph Extension . . . 16

3.3 Mathematical Properties and Theorems . . . 17
3.4 Mapping to Standard Graph Problems . . . 17
3.5 Handover Detection Flow . . . 17
3.6 Summary of Key Formulas . . . 17

4 ALGORITHM DESIGN AND SOLUTION APPROACH 20
4.1 Introduction . . . 20
4.2 Single Vehicle Detection Algorithm . . . 20
4.2.1 Algorithm Overview . . . 20
4.2.2 Preprocessing Phase . . . 21
4.2.3 Initial Clustering Phase . . . 22
4.2.4 Handover-Based Cluster Merging . . . 23
4.2.5 Decision Logic . . . 25
4.2.6 Complete Algorithm . . . 25
4.2.7 Worked Example: Step-by-Step Walkthrough . . . 25
4.2.8 Correctness Analysis . . . 25
4.3 Complexity Analysis . . . 26
4.4 Error Handling and Edge Cases . . . 26
4.4.1 Input Validation . . . 26
4.4.2 Edge Cases . . . 26
4.5 Algorithm Decision Flow . . . 28
4.6 Summary . . . 28

REFERENCES 28

LIST OF TABLES

1.1 Validation challenge: distinguishing legitimate tracking patterns from anomalies. Each row describes an observable pattern in the shipment's position history, the most likely root cause, a concrete example, and the expected validation outcome. 5

2.1 Index of notation and symbols 13

3.1 Mapping our problem to standard graph problems 18

4.1 Complexity summary 26
4.2 Edge case handling 27

LIST OF FIGURES

1.1 A typical GPS-based shipment tracking interface showing the trail of a shipment in transit 2
1.2 Scenario A: Two GPS devices incorrectly assigned to the same shipment ID in the configuration database 3
1.3 Impossible Teleportation: a sudden, physically unreachable jump in the position timeline caused by an interleaved ping from a second device at a distant location 4
1.4 Oscillating Locations: the recorded trajectory alternates between two distant geographic regions as pings from two devices (D1 and D2) interleave chronologically under the same shipment ID 4
1.5 Multi-vehicle anomaly illustration showing simultaneous GPS trails for the same shipment ID at distant locations, demonstrating the "impossible teleportation" and "oscillating locations" symptoms 6
1.6 System overview: end-to-end validation pipeline 7

2.1 Visual examples of the graph types discussed: directed graph, DAG, bipartite graph, tree, and connected components 10
2.2 Graph construction pipeline: from raw GPS data to validation result 13

3.1 Connected components in a graph, color-coded by cluster. Each component represents a group of positions that are mutually reachable, corresponding to positions from the same vehicle. 16
3.2 Handover detection decision flow for cluster pair (C_1, C_2) 19

4.1 Three-phase pipeline of the SingleVehicleCheck algorithm 21
4.2 Step-by-step walkthrough of the SingleVehicleCheck algorithm on concrete GPS data from Shanghai and Shenzhen. Both clusters meet the significance threshold, confirming multiple vehicles. The algorithm returns FALSE. 27
4.3 Complete decision flow of the SingleVehicleCheck algorithm 28

CHAPTER 1

PROBLEM STATEMENT

1.1 Introduction

Modern logistics and supply chain systems rely heavily on GPS-based shipment tracking to provide real-time visibility of goods in transit. The global Real-Time Transportation Visibility Platform (RTTVP) market, valued at approximately USD 3.2 billion in 2024 [1], underscores the scale of this dependence: leading platforms process billions of data points daily across millions of active shipments [2, 3]. This data forms the backbone of supply chain visibility, enabling businesses to track deliveries, estimate arrival times, optimize routes, and respond to disruptions.

However, the quality of this GPS data is frequently compromised. Data quality in shipment tracking is inherently multidimensional [4, 5, 6]: it spans positional accuracy (how closely reported coordinates match true locations), completeness (whether all expected pings are received), logical consistency (whether successive positions form a physically plausible trajectory), timeliness (whether data arrives within acceptable latency), and format validity (whether coordinates and timestamps conform to expected ranges and conventions). In real-world IoT deployments, these dimensions degrade due to GNSS signal obstruction, hardware faults, transmission latency, and data integration inconsistencies [7].

The root causes of degraded quality span both hardware and integration layers. On the hardware side, signal multipath in urban canyons, cold-start satellite acquisition delays, firmware bugs, and antenna degradation all introduce positional errors or reporting gaps [8, 9, 10]. A particularly insidious class of errors arises from coordinate system mismatches: the standard geographic convention (ISO 6709) represents a point as (latitude, longitude), whereas GeoJSON (RFC 7946) uses (longitude, latitude) [11]. When data flows between systems without proper transformation, coordinates are silently swapped, placing vehicles at entirely incorrect locations. On the integration side, incorrect device-to-shipment assignment, shipment ID collisions when merging feeds from multiple carriers [12], and system migration artifacts all corrupt data at the platform level. These failure modes map directly to the error scenarios described in Section 1.2.

Consider a realistic scenario illustrating how these issues compound. A refrigerated pharmaceutical shipment travelling across Europe experiences a tunnel-induced reporting gap, while a recent system migration introduces a coordinate swap for a subset of positions. The tracking platform, receiving this degraded data, triggers a false delay alert; the customer authorises an unnecessary backup shipment. Meanwhile, the swapped coordinates produce an anomalous teleportation event that corrupts the carrier's delivery metrics. Industry research estimates that poor data quality costs organisations millions annually [13], highlighting the need for robust, multi-dimensional validation.

This project focuses on one of the critical data quality problems encountered in production shipment tracking systems: **Multi-Vehicle Tracking Anomaly**, a single shipment being incorrectly tracked by

multiple vehicles simultaneously. We propose a unified graph-theoretic framework to model, detect, and resolve these anomalies.

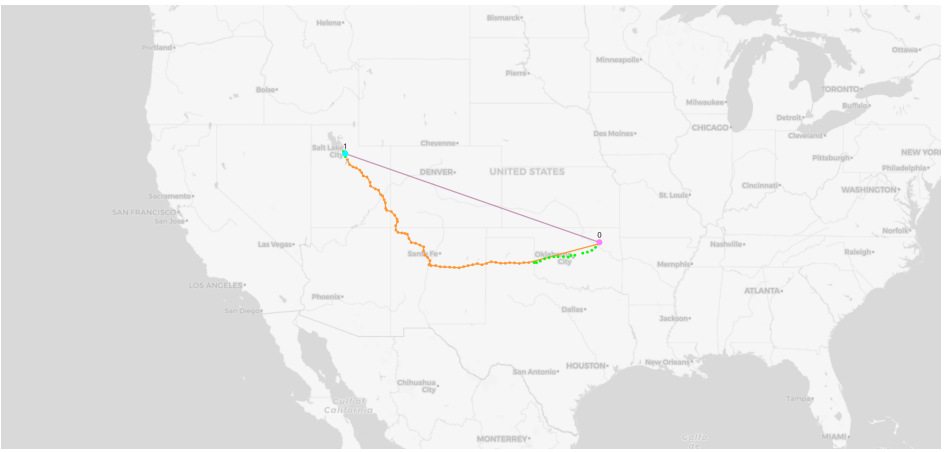


Figure 1.1: A typical GPS-based shipment tracking interface showing the trail of a shipment in transit

1.2 Problem: Shipment Tracked by Multiple Vehicles

1.2.1 Problem Description

In a correctly functioning tracking system, a shipment (such as a container, parcel, or pallet) should be associated with exactly one tracking device at any given time. The GPS positions reported for that shipment should represent the movement of a single physical vehicle carrying the goods.

When two or more GPS devices are erroneously associated with the same shipment ID, each device independently reports the position of the vehicle it is physically mounted on. Because these vehicles are travelling along different routes in different locations, the tracking platform receives two streams of coordinates that it merges into a single timeline under one shipment record. The result is a position history that no single physical object could have produced: the shipment appears to exist in two places at once, or to jump between distant locations faster than any vehicle could travel. The platform has no inherent way to know that the data originates from separate hardware sources; it simply timestamps and stores every incoming GPS ping against the shipment ID it was tagged with.

The following erroneous scenarios describe how this dual-device (or multi-device) binding occurs in practice:

Scenario A: Device Misconfiguration. Each GPS tracking device is bound to a shipment via a mapping record in the platform's configuration database. Misconfiguration occurs when this mapping is incorrectly established, causing multiple devices to reference the same shipment simultaneously [14, 15]. Common causes include a dispatcher entering the wrong shipment ID, bulk-upload spreadsheets with duplicated references, or a device-swap procedure where the original tracker is not deregistered [16].

For example, two trucks on different routes (Munich to Hamburg and Berlin to Frankfurt) are both assigned to `SHIP-12345`. The dashboard displays interleaved pings 250 km apart within seconds, producing an implied velocity of over 300,000 km/h. Figure 1.2 illustrates the database-level error.

Scenario B: Data Integration Errors. When integrating data from multiple carrier partners or legacy systems, shipment IDs may collide or be incorrectly mapped, causing two distinct shipments to merge into one record [17, 18]. Root causes include the absence of carrier-scoped namespaces (e.g., both Carrier A and Carrier B independently assign shipment number `1001`), collision-prone ID hashing, or legacy EDI

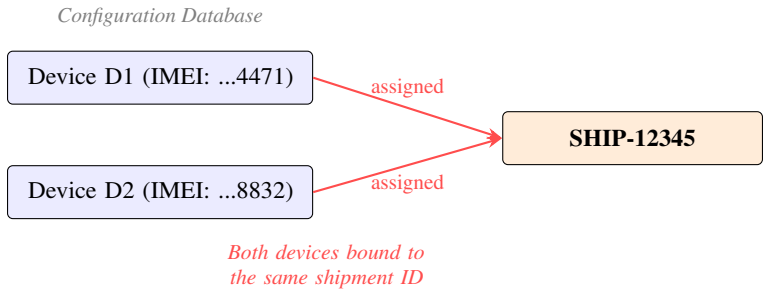


Figure 1.2: Scenario A: Two GPS devices incorrectly assigned to the same shipment ID in the configuration database

interfaces that transmit fixed-width codes without a carrier prefix [19]. The merged record overlays two unrelated trajectories, corrupting mileage-based billing and making SLA adjudication impossible.

Scenario C: Duplicate Device Transmissions. A malfunctioning device or network issue causes duplicate position records to be created, sometimes with slightly different coordinates or timestamps.

Scenario D: System Migration Issues. During system migrations or updates, historical data from old tracking devices may get incorrectly linked with new device data for the same shipment ID.

Impact of above Scenarios: When duplicate transmissions occur, the position history gains spurious entries that can scatter across the timeline. If the duplicates carry slightly perturbed coordinates (due to jitter introduced during re-transmission or buffering), the reachability analysis encounters pairs of positions that appear to originate from two distinct spatial locations at nearly the same instant. This artificially splits what should be a single connected component into two or more fragments, causing the algorithm to flag a legitimate single-vehicle journey as a multi-vehicle anomaly. Even when coordinates are identical, duplicate timestamps corrupt implied-speed calculations, inflate the total position count used for significance thresholds, and undermine jitter-window deduplication logic. In high-frequency tracking deployments, a single faulty network hop can inject hundreds of duplicate pings within minutes, overwhelming the noise-tolerance mechanisms designed for isolated GPS outliers.

1.2.2 Observable Symptoms

When a shipment is tracked by multiple vehicles, the position history exhibits characteristic patterns:

- **Impossible Teleportation:** Consecutive positions show the shipment at locations that are physically impossible to reach in the elapsed time. For example, a position in Shanghai at 10:00 AM followed by Shenzhen (1,200 km away) at 10:05 AM implies 14,400 km/h. Detection relies on computing the implied velocity $V_{\text{implied}} = d(p_i, p_{i+1})/(t_{i+1} - t_i)$ and comparing it against feasibility thresholds [20, 21, 22].

- **Oscillating Locations:** The shipment appears to "jump" back and forth between two or more distinct geographic areas, as positions from different vehicles interleave in the timeline, analogous to the "ping-pong handover" effect in cellular networks [23, 24].

- **Concurrent Distant Positions:** Multiple positions recorded at nearly the same timestamp show the shipment at vastly different locations simultaneously, proving that the data originates from at least two physical devices [25, 26].

Algorithmically, these symptoms can be resolved through spatiotemporal density clustering [27] and graph-based trajectory modelling [28].

Figures 1.3 and 1.4 visualise two of these symptoms as position-versus-time diagrams, contrasting the anomalous pattern with the expected single-vehicle trajectory.

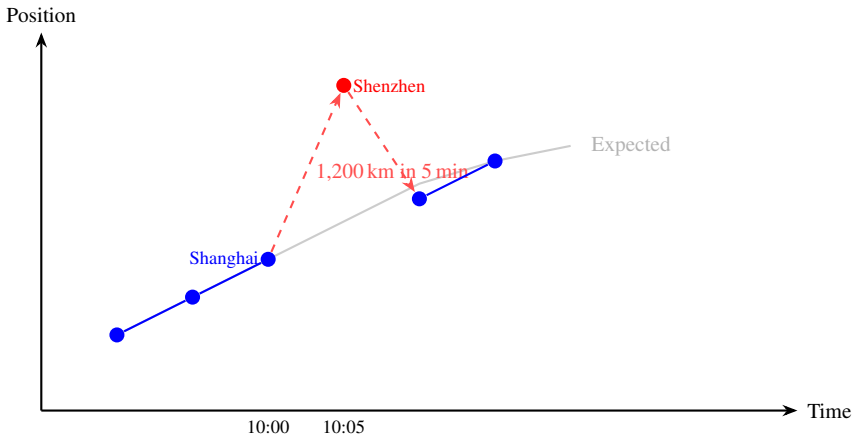


Figure 1.3: Impossible Teleportation: a sudden, physically unreachable jump in the position timeline caused by an interleaved ping from a second device at a distant location

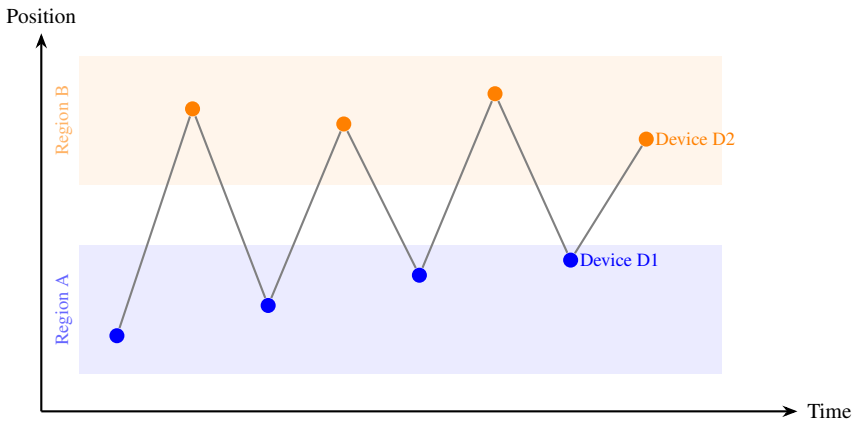


Figure 1.4: Oscillating Locations: the recorded trajectory alternates between two distant geographic regions as pings from two devices (D1 and D2) interleave chronologically under the same shipment ID

1.2.3 Legitimate Exception: Vehicle Handovers

Not all multi-location patterns indicate errors. A shipment may genuinely span two distant locations in rapid succession when it physically transfers between vehicles, such as a truck handing off to a ship at a port, a long-haul truck transferring to a local van at a logistics hub, or a relay swap at a highway point. The distinguishing characteristic is *spatial and temporal feasibility*: the two position clusters share a common meeting point, and the time gap between the last ping of Vehicle 1 and the first ping of Vehicle 2 is physically reasonable [29, 30]. In contrast, an erroneous multi-location pattern exhibits no such bridging point; positions jump between distant areas with no feasible intermediate transition.

1.2.4 The Validation Challenge

The system must distinguish between the cases shown in Table 1.1.

1.2.5 Business Impact

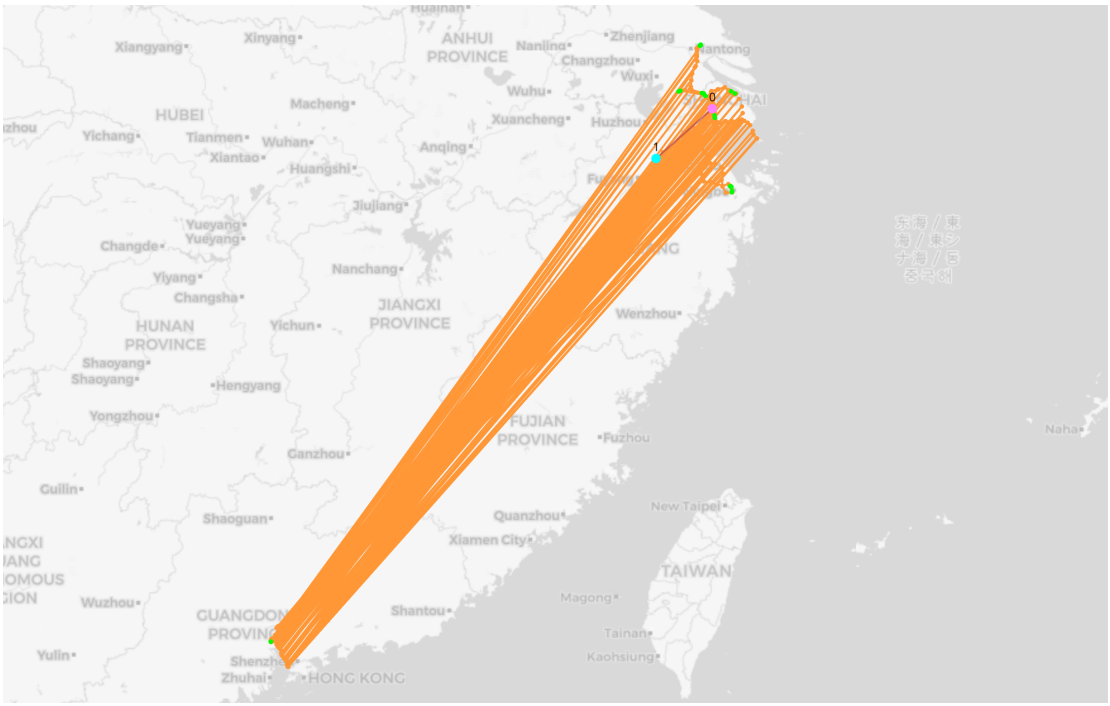
Failure to detect multi-vehicle tracking errors carries severe financial consequences. The American Transportation Research Institute reports that the U.S. for-hire trucking industry lost 135.9 million hours and $11.5 billion in revenue to detention in 2023 alone [31], while nine major ocean carriers collected $12.9 billion in demurrage and detention fees between 2020 and 2023 [32]. Minor tracking delays

Table 1.1: Validation challenge: distinguishing legitimate tracking patterns from anomalies. Each row describes an observable pattern in the shipment's position history, the most likely root cause, a concrete example, and the expected validation outcome.

Observable Pattern	**Likely Cause**	**Example**	**Expected Result**
All positions reachable in sequence at plausible speeds	Single vehicle, normal tracking	Truck moves Mumbai → Pune at 80 km/h; every consecutive pair of pings is reachable within the elapsed time	VALID
Position gap followed by resumption at a common location	Legitimate vehicle handover	Last ping from Truck A at a port at 09:00; first ping from Ship B at the same port at 11:00; the 2-hour gap is consistent with a loading operation	VALID
Positions at two distant locations at the same timestamp	Multi-device error (Scenario A or B)	Ping in Shanghai and ping in Shenzhen (1,200 km apart) both stamped 10:00 AM, physically impossible for one shipment	INVALID
Positions alternating between two distant regions over time	Data integration error or device misconfiguration	Shipment oscillates between the Munich–Hamburg corridor and the Berlin–Frankfurt corridor every few minutes as two devices interleave	INVALID
A small number of unreachable positions among many consistent ones	GPS jitter, momentary multipath, or brief signal anomaly	99 of 100 pings form a smooth highway trajectory; one ping places the truck 500 m off-road due to urban-canyon multipath	VALID (with tolerance)

cascade into production stoppages, SLA penalties, and costly contingency measures such as expedited air freight [33]. Specific consequences include:

- **Customer Confusion:** Tracking pages show shipments jumping between locations erratically.
- **Incorrect ETAs:** Arrival time predictions become meaningless when location data is inconsistent, triggering premature or delayed dock-door scheduling and labour allocation.
- **Billing Disputes:** Mileage-based billing calculations are corrupted by phantom movements; industry data indicates that while 75% of detention hours are invoiced, only 55% are ultimately paid, leaving billions locked in dispute resolution [31].
- **Operational Blindness:** Supply chain managers cannot trust the tracking data for decision-making, forcing reactive "Where Is My Order" firefighting.
- **SLA Violations:** False delay alerts or missed actual delays due to corrupted position data, leading to unnecessary backup shipments and expedited freight premiums.



(a) Before: conflicting GPS trails from two devices assigned to the same shipment

(b) After: trails separated and anomaly identified

Figure 1.5: Multi-vehicle anomaly illustration showing simultaneous GPS trails for the same shipment ID at distant locations, demonstrating the “impossible teleportation” and “oscillating locations” symptoms

1.3 System Overview

Figure 1.6 illustrates the end-to-end pipeline of the proposed validation framework.

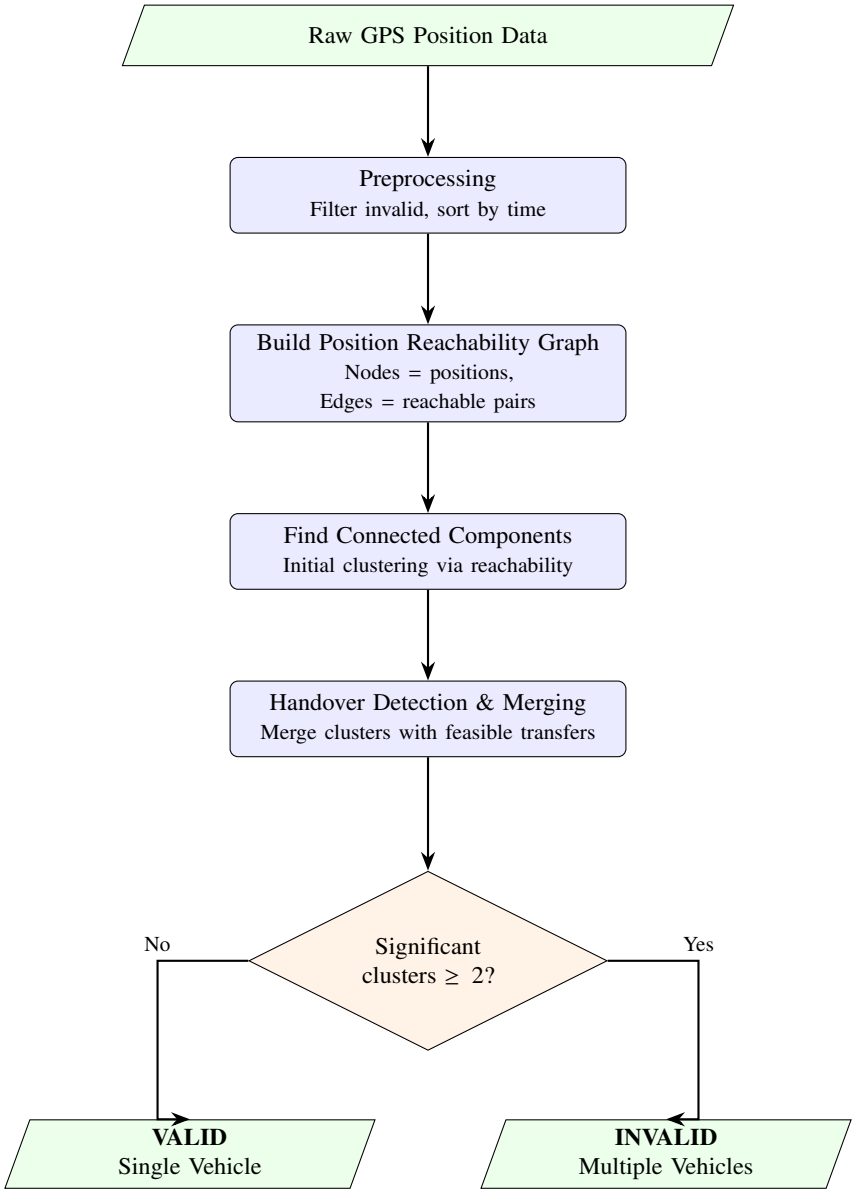


Figure 1.6: System overview: end-to-end validation pipeline

The pipeline operates as follows:

1. **Preprocessing.** The raw GPS position stream is cleaned and sorted chronologically. Invalid records (such as positions with out-of-range coordinates, null timestamps, or duplicate entries) are discarded. The output is an ordered position sequence $P = \{p_1, p_2, \ldots, p_n\}$ with $t_1 \leq t_2 \leq \cdots \leq t_n$, ready for graph construction.

2. **Build Position Reachability Graph.** Each position becomes a node, and a directed edge is added between two consecutive positions whenever the Haversine distance between them does not exceed $V_{\max} \times \Delta t$ (i.e., the movement is physically feasible at the maximum allowed speed). The resulting graph encodes all feasible single-vehicle transitions.

3. **Find Connected Components.** The reachability graph is partitioned into connected components

using an online clustering algorithm. Each component groups positions that are mutually reachable through a chain of feasible movements, representing a candidate single-vehicle trajectory. Multiple components indicate that some positions cannot be connected by any physically plausible path.

4. **Handover Detection & Merging.** Not all distinct components imply separate vehicles; a shipment may legitimately transfer between vehicles (e.g., at a warehouse or cross-dock). The algorithm tests whether the boundary positions of two clusters (last k positions of one, first k of the next) are spatially and temporally close enough to represent a physical handover. Clusters connected by valid handovers are merged into a single super-cluster.

5. **Decision: Significant Clusters ≥ 2?** After merging, small residual clusters caused by GPS jitter or isolated outlier positions are filtered out using a significance threshold. If two or more clusters of substantial size remain, the shipment is confirmed to have been tracked by multiple vehicles simultaneously, and the result is **INVALID**. If only one significant cluster remains (possibly formed by merging several handover-connected components), the tracking data is consistent with a single vehicle and the result is **VALID**.

CHAPTER 2

GRAPH THEORY BACKGROUND

2.1 Graph Theory Fundamentals

The definitions and algorithmic foundations in this chapter follow the standard treatment in Cormen et al. [34].

2.1.1 Basic Definitions

Definition 1 (Graph) *A graph G is an ordered pair $G = (V, E)$ where:*

- *V is a set of vertices (also called nodes),*
- *E is a set of edges, where each edge connects two vertices.*

Definition 2 (Directed Graph) *A directed graph (or digraph) is a graph where each edge has a direction. An edge from vertex u to vertex v is denoted as (u, v) or $u \to v$, and is distinct from (v, u).*

Definition 3 (Weighted Graph) *A weighted graph assigns a numerical value (weight) to each edge. Formally, it includes a weight function $w : E \to \mathbb{R}$ that maps edges to real numbers.*

Definition 4 (Adjacency) *Two vertices u and v are adjacent if there exists an edge connecting them. In a directed graph, u is adjacent to v if edge (u, v) exists.*

Definition 5 (Degree) *The degree of a vertex v, denoted $\deg(v)$, is the number of edges incident to v. In directed graphs:*

- ***In-degree:*** *Number of edges coming into v*
- ***Out-degree:*** *Number of edges going out from v*

2.1.2 Paths and Connectivity

Definition 6 (Path) *A path in a graph is a sequence of vertices $(v_1, v_2, \ldots, v_k)$ such that each consecutive pair is connected by an edge. The length of the path is $k - 1$ (number of edges).*

Definition 7 (Connected Graph) *An undirected graph is connected if there exists a path between every pair of vertices.*

Definition 8 (Weakly Connected) *A directed graph is weakly connected if replacing all directed edges with undirected edges results in a connected graph.*

Definition 9 (Strongly Connected) *A directed graph is strongly connected if there exists a directed path from every vertex to every other vertex.*

Definition 10 (Connected Component) *A connected component of a graph is a maximal subgraph in which every pair of vertices is connected by a path. A graph can have multiple connected components.*

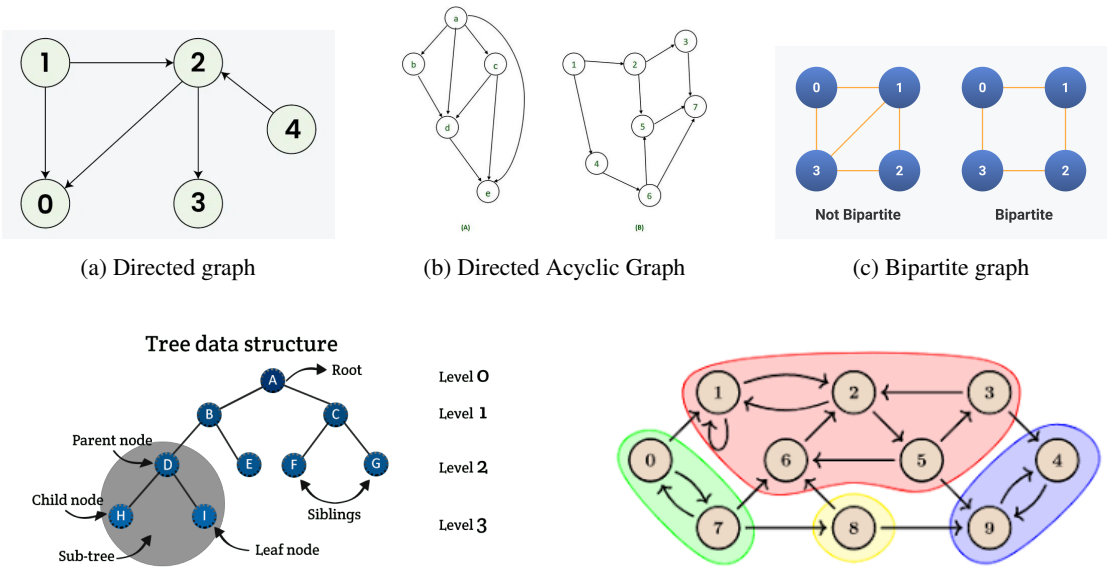
2.1.3 Special Graph Types

Definition 11 (Directed Acyclic Graph (DAG)) *A directed acyclic graph is a directed graph with no directed cycles. That is, there is no sequence of edges that starts and ends at the same vertex.*

Definition 12 (Bipartite Graph) *A bipartite graph is a graph whose vertices can be partitioned into two disjoint sets U and V such that every edge connects a vertex in U to a vertex in V. No edge connects vertices within the same set.*

Definition 13 (Complete Graph) *A complete graph K_n is a graph with n vertices where every pair of vertices is connected by an edge.*

Definition 14 (Tree) *A tree is a connected acyclic graph. A tree with n vertices has exactly $n - 1$ edges.*



(a) Directed graph (b) Directed Acyclic Graph (c) Bipartite graph

(d) Tree (e) Connected components

Figure 2.1: Visual examples of the graph types discussed: directed graph, DAG, bipartite graph, tree, and connected components

2.1.4 Graph Representations

Adjacency Matrix. For a graph with n vertices, the adjacency matrix A is an $n \times n$ matrix where:

- $A[i][j] = 1$ if edge (i, j) exists,
- $A[i][j] = 0$ otherwise.

For weighted graphs, $A[i][j]$ contains the edge weight instead of 1.
Space complexity: $O(n^2)$. Edge lookup: $O(1)$. Finding all neighbors: $O(n)$.

Adjacency List. Each vertex maintains a list of its adjacent vertices.

Space complexity: $O(n + m)$ where m is the number of edges. Edge lookup: $O(\text{degree})$. Finding all neighbors: $O(\text{degree})$.

Implicit Representation. For some graphs, edges are not stored explicitly but computed on demand using a predicate function. This is memory-efficient when the graph is dense or the edge predicate is simple. Our solution uses implicit representation where edges are computed via the reachability predicate.

2.2 Spatial Data Concepts

2.2.1 Geographic Coordinate Systems

Latitude and Longitude.

- **Latitude** (ϕ): Angular distance north or south of the equator, ranging from -90 to $+90$ degrees.
- **Longitude** (λ): Angular distance east or west of the Prime Meridian, ranging from -180 to $+180$ degrees.

Coordinate Conventions.

- Standard: (latitude, longitude), used by most mapping applications.
- GeoJSON: (longitude, latitude), used in GeoJSON format and some APIs.

This inconsistency is a root cause of coordinate swap errors.

2.2.2 Distance Calculations

The central question in our anomaly detection framework is deceptively simple: *could a truck have physically travelled from GPS ping p_1 to the next ping p_2 in the time that elapsed between them?* Answering this requires computing the geographic distance between two coordinate pairs reported by the tracking device. Because the Earth is approximately spherical, the shortest path between two surface points is not a straight line through the interior but an arc along the surface, known as a *great circle*. The Haversine formula computes the length of this arc.

Analogy. Imagine stretching a taut string between two pins on a globe: the string follows the surface curvature, and its length is the great-circle distance, the minimum distance a vehicle must cover between the two locations. The Haversine formula computes this arc length.

Haversine Formula. Given two GPS positions $p_1 = (\phi_1, \lambda_1)$ and $p_2 = (\phi_2, \lambda_2)$ in radians, the great-circle distance is computed as follows. Let $R = 6{,}371$ km denote the mean Earth radius:

$$\Delta\phi = \phi_2 - \phi_1, \qquad \Delta\lambda = \lambda_2 - \lambda_1$$

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right)$$

$$c = 2 \cdot \arcsin\left(\sqrt{a}\right), \qquad d = R \cdot c$$

Here $\Delta\phi$ and $\Delta\lambda$ capture the north–south and east–west angular separations. The term a combines these while correcting for meridian convergence near the poles via $\cos(\phi)$: a 1° longitude difference spans ≈ 111 km at the equator but only ≈ 28 km near the Arctic. The arc c converts the angular separation to a fraction of the Earth's circumference, yielding surface distance d.

Worked example. A shipment reports two consecutive pings:

- p_1: Mumbai (19.076°N, 72.878°E) at 10:00 AM

- p_2: Pune (18.520°N, 73.856°E) at 12:30 PM

Applying the Haversine formula yields $d \approx 149$ km. The elapsed time is 2.5 hours, giving an implied speed of $149/2.5 = 59.6$ km/h, well below our 250 km/h threshold. The reachability predicate returns `TRUE`: this pair of pings is consistent with a single truck travelling on the Mumbai–Pune Expressway.

Now consider a conflicting ping from a second device: p_2': Chennai (13.083°N, 80.270°E) at 10:15 AM. The Haversine distance from Mumbai to Chennai is $\approx 1{,}028$ km, and the elapsed time is just 15 minutes (0.25 h). The implied speed is $1{,}028/0.25 = 4{,}112$ km/h, clearly impossible for any ground vehicle. The reachability predicate returns `FALSE`, and this ping becomes a candidate for the multi-vehicle anomaly described in Chapter 1.

Accuracy. The Haversine formula assumes a perfectly spherical Earth. The resulting error is less than 0.3% compared to ellipsoidal calculations, increasing to approximately 0.55% for trans-equatorial distances.

Vincenty Formula. Vincenty [35] developed a more accurate distance calculation using an ellipsoidal Earth model (WGS-84), achieving accuracy to within 0.5 mm. It is computationally more expensive due to its iterative nature. For our application, the Haversine formula provides sufficient accuracy with better performance, as positional errors from GPS hardware (3–10 m under open sky [7]) far exceed the sub-metre discrepancy between the two formulae.

2.2.3 Speed Constraints and Reachability

We use 250 km/h (69.4 m/s) as a conservative upper bound for ground transport. EU Directive 2002/85/EC mandates speed limiters set to 90 km/h for heavy goods vehicles [36], and practical motorway limits across Europe range from 80 to 100 km/h for trucks. Our threshold of 250 km/h is deliberately generous to accommodate GPS timestamp inaccuracies, brief high-speed rail segments in intermodal chains, and the need to avoid false positives on legitimate data.

Reachability Predicate. Position p_2 is reachable from p_1 if:

$$\text{distance}(p_1, p_2) \leq \text{MAX_SPEED} \times (p_2.\text{time} - p_1.\text{time})$$

This forms the basis for edge creation in our reachability graph.

2.3 Notation Index

Table 2.1 summarizes the mathematical notation used throughout this report.

2.4 Graph Construction Overview

Figure 2.2 illustrates how raw GPS data is transformed into graph structures for analysis.

Table 2.1: Index of notation and symbols

Symbol	Description
$G = (V, E)$	Graph with vertex set V and edge set E
$p_i = (lat, lon, t)$	Position tuple: latitude, longitude, timestamp
$P = \{p_1, \ldots, p_n\}$	Position sequence sorted by timestamp
$V_{\max}$	Maximum feasible speed (69.4 m/s = 250 km/h)
$\text{haversine}(p_i, p_j)$	Great-circle distance between two positions
$\text{reachable}(p_i, p_j)$	Reachability predicate: dist $\leq V_{\max} \cdot \Delta t$
C_k	Cluster (connected component) of positions
$C = \{C_1, \ldots, C_m\}$	Set of all clusters
τ	Jitter tolerance window (1800 s = 30 min)
$W(p_j, C, \tau)$	Jitter window: recent positions within τ seconds
T_{HANDOVER}	Handover time window (28800 s = 8 hours)
$\text{first}_k(C)$, $\text{last}_k(C)$	First/last k boundary positions of cluster C
$H = (C, E_H)$	Handover Graph over clusters
α	Minimum cluster percentage threshold (0.10)
$s_{\min}$	Minimum cluster size threshold (5)
R	Earth's radius (6,371,000 m)

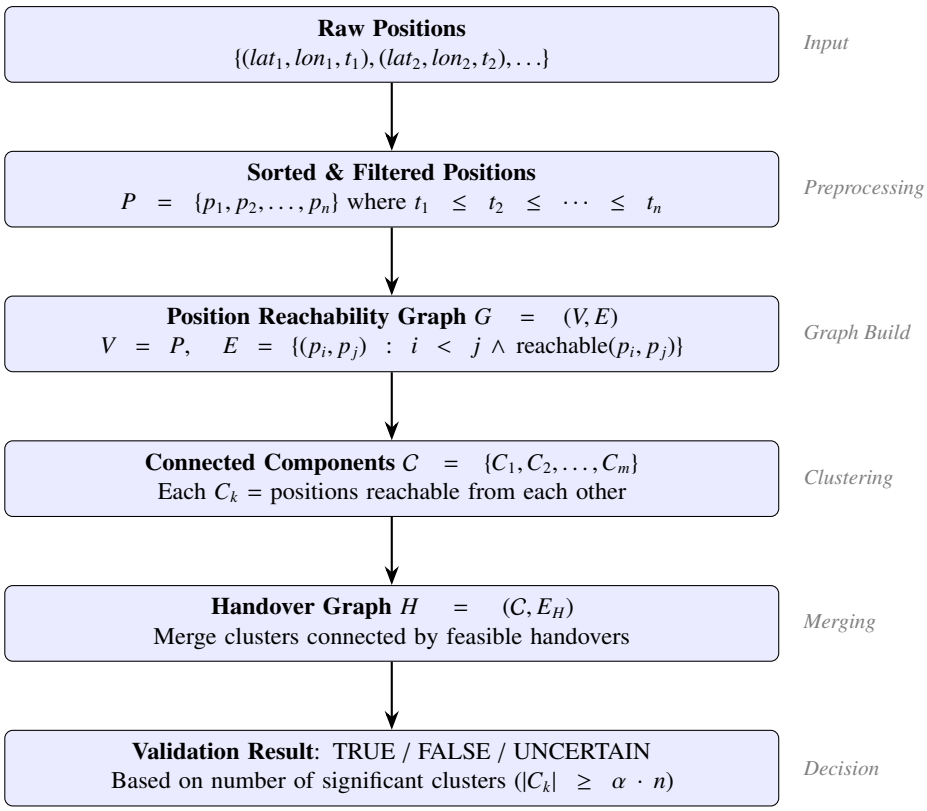


Figure 2.2: Graph construction pipeline: from raw GPS data to validation result

CHAPTER 3

GRAPH FORMULATION

3.1 Introduction

This chapter presents the formal graph-theoretic models for our shipment tracking validation problems. We transform raw GPS position data into structured graph representations that capture the essential relationships between data points. This abstraction enables us to leverage well-established graph algorithms and provides mathematical rigor to our validation approach.

The key insight is that the problems can be viewed through the lens of connectivity: Are all positions connected through feasible physical movements? The graph algorithms we rely on, namely online single-pass clustering for connected component discovery and iterative pairwise union for cluster merging, build on well-established foundations in the algorithmic literature [34].

3.2 Position Reachability Graph

3.2.1 Intuition

Consider a shipment's position history as a sequence of events. If a single vehicle is tracking the shipment, then each position should be "reachable" from previous positions given the physical constraints of movement. Unreachable positions suggest either GPS errors (if few) or multiple vehicles (if many form distinct patterns).

We model this as a directed graph where:

- Each position is a node.
- An edge exists between two positions if one can physically reach the other.
- Connected components represent groups of positions that could belong to the same vehicle.

3.2.2 Formal Definitions

Definition 15 (Position) *A position p is a tuple (lat, lon, t) where:*

- *lat is the latitude in decimal degrees, $lat \in [-90, 90]$,*
- *lon is the longitude in decimal degrees, $lon \in [-180, 180]$,*
- *t is the timestamp (Unix epoch or datetime).*

Definition 16 (Position Sequence) *A position sequence P for a shipment is an ordered set $P = \{p_1, p_2, \ldots, p_n\}$ sorted by timestamp such that $t_1 \leq t_2 \leq \cdots \leq t_n$.*

Definition 17 (Reachability Predicate) *Two positions p_i and p_j (where $t_i < t_j$) satisfy the reachability predicate, denoted* $\text{reachable}(p_i, p_j)$*, if and only if*

$$\text{haversine}(p_i, p_j) \leq V_{\max} \cdot (t_j - t_i),$$

where:

- $\text{haversine}(p_i, p_j)$ *is the great-circle distance in meters,*
- $V_{\max}$ *is the maximum feasible speed (69.4 m/s = 250 km/h),*
- $(t_j - t_i)$ *is the time difference in seconds.*

Definition 18 (Position Reachability Graph) *Given a position sequence P, the Position Reachability Graph is a directed graph $G = (V, E)$ where:*

- $V = P$ *(each position is a vertex),*
- $E = \{(p_i, p_j) : i < j \wedge \text{reachable}(p_i, p_j)\}$.

Edges are directed from earlier positions to later positions.

3.2.3 Graph Properties

Property 3.1 (Directed Acyclic Graph) *The Position Reachability Graph G is always a DAG.*

Proof. By construction, edges only go from positions with earlier timestamps to positions with later timestamps. Since $t_i < t_j$ for any edge (p_i, p_j), there cannot exist a cycle. A cycle would require an edge from a later timestamp back to an earlier timestamp, which is impossible by our edge definition.

Property 3.2 (Temporal Ordering) *The topological order of G corresponds to the chronological order of positions.*

Property 3.3 (Transitivity) *Reachability is not necessarily transitive. If* $\text{reachable}(p_1, p_2)$ *and* $\text{reachable}(p_2, p_3)$*, it does not imply* $\text{reachable}(p_1, p_3)$.

Example: p_1 at Shanghai, $t = 0$; p_2 at Hangzhou (180 km away), $t = 2$ hours (reachable at 90 km/h); p_3 at Ningbo (additional 150 km), $t = 3$ hours (reachable from p_2 at 150 km/h). Then p_1 to p_3 is 330 km in 3 hours = 110 km/h (still reachable). With tighter time constraints, transitivity can fail.

3.2.4 Extended Reachability with GPS Jitter Tolerance

Real GPS data contains noise. A position might appear slightly unreachable from its immediate predecessor due to GPS jitter, even though it belongs to the same vehicle. We extend reachability to check multiple recent positions.

Definition 19 (Jitter Window) *The jitter window $W(p_j, C, \tau)$ for position p_j with respect to cluster C and time threshold τ is*

$$W(p_j, C, \tau) = \{p_i \in C : (t_j - t_i) \leq \tau\},$$

the set of positions in cluster C that are within τ seconds before p_j.

Definition 20 (Extended Reachability) *Position p_j is reachable from cluster C with jitter tolerance if*

$$\text{reachable_extended}(p_j, C) = \exists p_i \in W(p_j, C, \tau) : \text{reachable}(p_i, p_j),$$

where $\tau = 1800$ seconds (30 minutes) in our implementation.

Definition 21 (Extended Position Reachability Graph) *The Extended Position Reachability Graph $G' = (V, E')$ incorporates jitter tolerance: $V = P$, and*

$$E' = \{(p_i, p_j) : i < j \wedge (\text{reachable}(p_i, p_j) \vee \text{exists_jitter_path}(p_i, p_j))\},$$

where exists_jitter_path *checks if p_j can be reached through recent positions.*

3.2.5 Handover Graph Extension

To handle legitimate vehicle handovers, we introduce a second-level graph.

Definition 22 (Cluster) *A cluster C is a maximal set of positions that form a connected component in the Position Reachability Graph.*

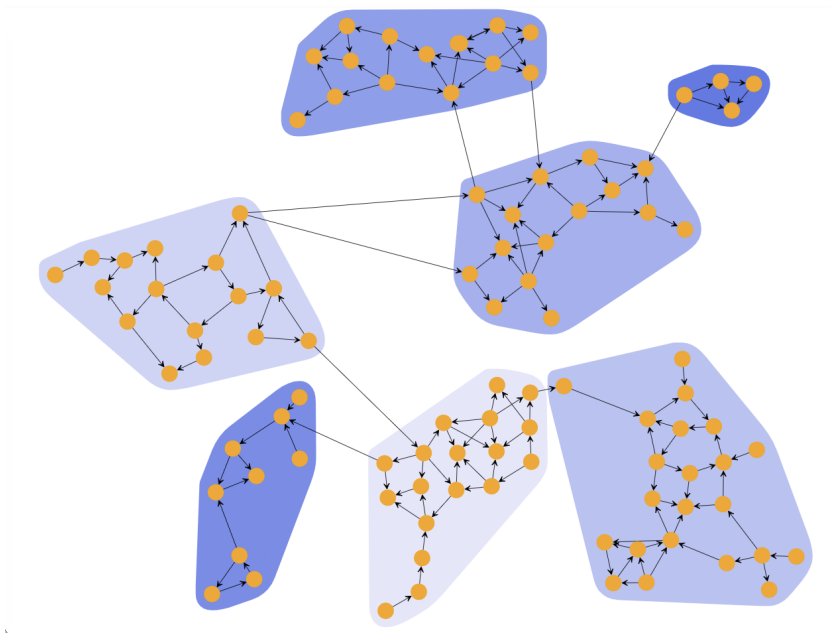


Figure 3.1: Connected components in a graph, color-coded by cluster. Each component represents a group of positions that are mutually reachable, corresponding to positions from the same vehicle.

Definition 23 (Cluster Boundary) *For a cluster C:*

- ***Entry boundary:*** $\text{first}_k(C)$ = *first k positions by timestamp,*
- ***Exit boundary:*** $\text{last}_k(C)$ = *last k positions by timestamp,*

where $k = 5$ in our implementation.

Definition 24 (Handover Predicate) *Two clusters C_1 and C_2 (where C_1 ends before C_2 starts) satisfy the handover predicate if*

$$\text{handover}(C_1, C_2) = \exists p_i \in \text{last}_k(C_1),\ p_j \in \text{first}_k(C_2) : |t_i - t_j| \leq T_{\text{HANDOVER}} \wedge \text{reachable}(p_i, p_j),$$

where $T_{\text{HANDOVER}} = 28800$ *seconds (8 hours).*

Definition 25 (Handover Graph) *Given clusters* $C = \{C_1, C_2, \ldots, C_m\}$*, the Handover Graph is* $H = (C, E_H)$ *where vertices are clusters and* $E_H = \{(C_i, C_j) : \text{handover}(C_i, C_j)\}$.

Definition 26 (Merged Cluster Graph) *The Merged Cluster Graph is the transitive closure of the Handover Graph, where connected clusters are merged into single super-clusters.*

3.3 Mathematical Properties and Theorems

Theorem 3.4 (Single Component Implies Single Vehicle) *If the Position Reachability Graph G has exactly one weakly connected component, then all positions are consistent with single-vehicle tracking.*

Proof. A single connected component means every position is reachable from every other position through some path. Since each edge represents a feasible physical movement, the entire trajectory represents feasible single-vehicle movement.

Theorem 3.5 (Multiple Large Components Implies Multiple Vehicles) *If G has two or more connected components, each containing at least 10% of total positions, then multiple vehicles are tracking the shipment.*

Proof. Two separate components with significant positions cannot both be GPS noise (which would be small, isolated clusters). The lack of edges between components means no feasible path exists between them, implying physically separate vehicles.

Theorem 3.6 (Handover Preserves Validity) *If two components* C_1 *and* C_2 *can be merged via a valid handover, the merged component represents valid single-vehicle tracking (with transfer).*

Proof. The handover predicate ensures spatial and temporal proximity at the transfer point, consistent with a physical handover event.

3.4 Mapping to Standard Graph Problems

3.5 Handover Detection Flow

Figure 3.2 shows the decision process for determining whether two clusters represent a legitimate vehicle handover.

3.6 Summary of Key Formulas

The following equations summarize the core mathematical foundations of the framework:

$$d(p_i, p_j) = 2R \cdot \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_i)\cos(\phi_j)\sin^2\left(\frac{\Delta\lambda}{2}\right)}\right) \tag{3.1}$$

$$\text{reachable}(p_i, p_j) \iff d(p_i, p_j) \leq V_{\max} \cdot (t_j - t_i) \tag{3.2}$$

$$\text{reachable_ext}(p_j, C) \iff \exists\, p_i \in W(p_j, C, \tau) : \text{reachable}(p_i, p_j) \tag{3.3}$$

Table 3.1: Mapping our problem to standard graph problems

Our Problem	**Standard Graph Problem**	**How We Apply It**
Initial clustering	Connected component finding	Each GPS position is a node in the Position Reachability Graph G. An edge (p_i, p_j) exists iff reachable(p_i, p_j) holds. Our online clustering algorithm (Algorithm 2) processes positions chronologically and assigns each to the first existing cluster whose recent positions satisfy the reachability predicate; if none match, a new cluster is created. The resulting clusters are equivalent to the connected components of G.
Cluster merging	Iterative pairwise union	The Handover Graph $H = (C, E_H)$ treats clusters as nodes. Algorithm 6 iterates over all cluster pairs, tests the handover predicate via CanHandover (Algorithm 7), and merges any connected pair. The loop repeats until no further merges occur, effectively computing the connected components of H and collapsing each into a single super-cluster.
Significant cluster detection	Component size thresholding	After merging, each remaining cluster is tested against the significance criterion: $\lvert C\rvert \geq \max(s_{\min},\ \alpha \cdot n)$ (Algorithm 9). Clusters below this threshold are classified as GPS jitter or outlier noise and discarded. If two or more clusters exceed the threshold, the algorithm returns `FALSE` (multiple vehicles detected).
Handover detection	Boundary reachability testing	For each cluster pair (C_1, C_2), Algorithm 7 extracts the last k positions of C_1 (last_k) and the first k positions of C_2 (first_k). It tests whether any exit–entry pair satisfies both the time constraint ($\lvert t_j - t_i\rvert \leq T_H$) and the reachability predicate. A match indicates a feasible physical transfer: the shipment changed vehicles at a common location within a plausible time window.

$$\text{handover}(C_1, C_2) \iff \exists\, p_i \in \text{last}_k(C_1),\ p_j \in \text{first}_k(C_2) : |t_i - t_j| \leq T_H \ \wedge\ \text{reachable}(p_i, p_j) \tag{3.4}$$

$$\text{significant}(C) \iff |C| \geq \max(s_{\min},\ \alpha \cdot n) \tag{3.5}$$

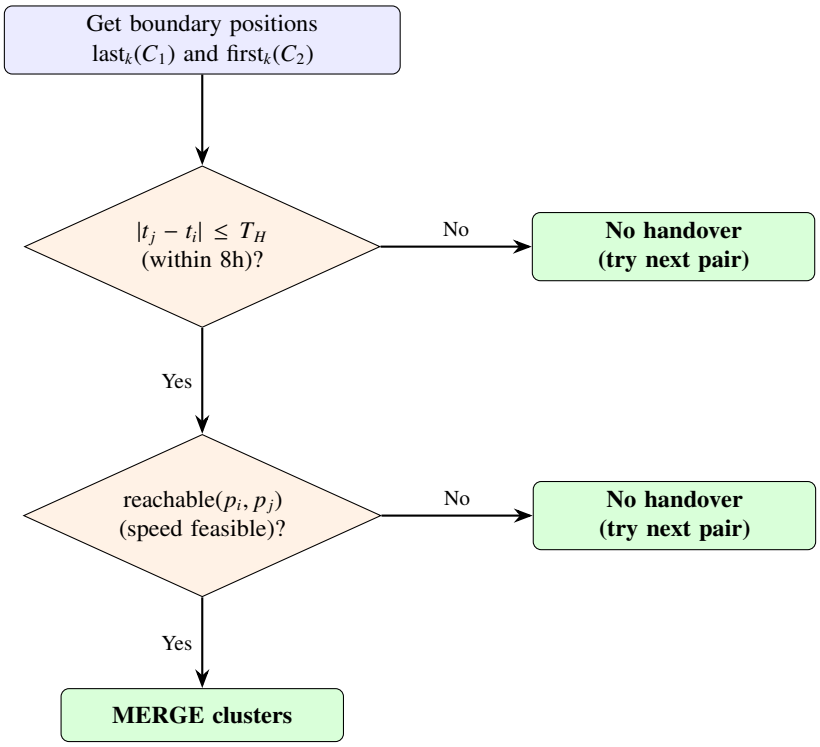


Figure 3.2: Handover detection decision flow for cluster pair (C_1, C_2)

CHAPTER 4

ALGORITHM DESIGN AND SOLUTION APPROACH

4.1 Introduction

This chapter presents the detailed algorithms for solving the shipment tracking validation problem. Building on the graph formulations from Chapter 3 and standard graph traversal techniques [34], we develop step-by-step procedures with pseudocode, analyze their correctness, and evaluate computational complexity.

The algorithms are designed with the following principles:

- **Correctness:** Produce accurate validation results based on graph properties.
- **Efficiency:** Process large position datasets in reasonable time.
- **Robustness:** Handle edge cases and noisy real-world data.
- **Interpretability:** Provide diagnostic information for debugging.

4.2 Single Vehicle Detection Algorithm

4.2.1 Algorithm Overview

The SingleVehicleCheck algorithm addresses the following decision problem: given a chronological sequence of GPS positions recorded under a single shipment identifier, determine whether the positions are consistent with tracking by a single physical vehicle, or whether they contain evidence of multiple distinct vehicles reporting simultaneously. The algorithm accepts a raw position sequence as input and produces a ternary validation result: `TRUE` (single vehicle), `FALSE` (multiple vehicles detected), or `UNCERTAIN` (insufficient data to conclude), along with diagnostic metadata describing the cluster structure.

The algorithm operates as a three-phase pipeline, illustrated in Figure 4.1. Each phase transforms the data into a progressively higher-level representation: raw coordinates are first cleaned into a valid temporal sequence, then partitioned into spatially coherent groups, and finally evaluated against significance thresholds to yield the validation verdict.

- **INPUT:** Raw position sequence $P = \{p_1, p_2, \ldots, p_n\}$, where each $p_i = (lat_i, lon_i, t_i)$
- **OUTPUT:** ValidationResult $\in \{\texttt{TRUE}, \texttt{FALSE}, \texttt{UNCERTAIN}\}$ with diagnostic metadata

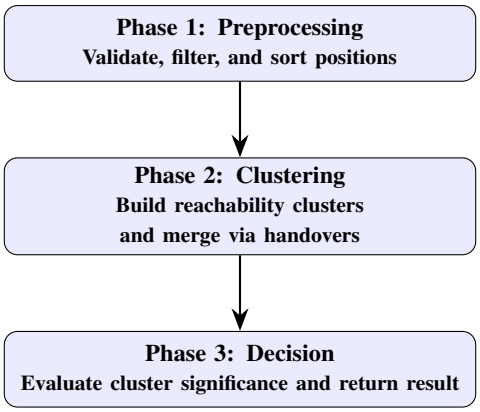


Figure 4.1: Three-phase pipeline of the SingleVehicleCheck algorithm

Phase 1: Preprocessing. The raw GPS payload is sanitised by discarding positions with null, NaN, or out-of-range coordinates (latitude $\notin [-90, 90]$, longitude $\notin [-180, 180]$), then sorted by timestamp to produce a clean sequence P'.

Phase 2: Clustering. The cleaned sequence P' is processed by an online clustering algorithm (Algorithm 2). Each position is tested against recent members of existing clusters using the Haversine-based reachability predicate with a jitter window of $\tau = 1800\,\text{s}$; reachable positions join the cluster, otherwise a new cluster is created. A second stage (Algorithm 6) then merges cluster pairs connected by feasible handovers (temporal gap $\leq T_H = 8$ hours and reachability satisfied).

Phase 3: Decision. The final cluster set C is evaluated against the significance criterion $|C| \geq \max(s_{\min},\ \alpha \cdot n)$ with $s_{\min} = 5$ and $\alpha = 0.10$ (Algorithm 9). Zero or one significant cluster yields `TRUE`/`UNCERTAIN`; two or more yields `FALSE` (multiple vehicles detected).

4.2.2 Preprocessing Phase

Algorithm 1: Position Preprocessing

Input: Raw position sequence P

Output: Cleaned, sorted position sequence P'

```
P' ← ∅;
foreach position p ∈ P do
    if p.lat or p.lon or p.time is NULL then
        skip;
    if isNaN(p.lat) or isNaN(p.lon) then
        skip;
    if p.lat ∉ [−90, 90] or p.lon ∉ [−180, 180] then
        skip;
    Append p to P';
Sort P' by timestamp ascending;
if |P'| = 0 then
    return empty list;
return P';
```

Complexity: Filtering $O(n)$, sorting $O(n \log n)$. Total: $O(n \log n)$.

4.2.3 Initial Clustering Phase

The initial clustering phase builds connected components using an online algorithm that processes positions chronologically.

Algorithm 2: Build Initial Clusters

Input: Sorted position sequence $P = \{p_1, p_2, \ldots, p_n\}$

Output: List of clusters $C = \{C_1, C_2, \ldots, C_m\}$

Constants: $V_{\max} = 69.4$ m/s, $JITTER_WIN = 1800$ s, $JITTER_CNT = 3$

```
clusters ← ∅;
for i ← 1 to n do
    p ← P[i];
    assigned ← FALSE;
    for j ← |clusters| to 1 do
        C ← clusters[j];
        recent ← GetRecentPositions(C, p.time, JITTER_WIN, JITTER_CNT);
        foreach q ∈ recent do
            if IsReachable(q, p, V_max) then
                Append p to C;
                C.latestTime ← p.time;
                assigned ← TRUE;
                break;
        if assigned then
            break;
    if ¬assigned then
        Create new cluster with {p} and append to clusters;
return clusters;
```

Algorithm 3: IsReachable

Input: Positions p_1, p_2; maximum speed *maxSpeed*

Output: Boolean indicating physical reachability

```
dist ← HaversineDistance(p_1, p_2);
Δt ← |p_2.time − p_1.time|;
if Δt = 0 then
    return dist < 100;
return (dist / Δt) ≤ maxSpeed;
```

Algorithm 4: HaversineDistance

Input: Positions $p_1 = (lat_1, lon_1)$, $p_2 = (lat_2, lon_2)$

Output: Great-circle distance in meters

```
R ← 6,371,000 ;                                   // Earth's radius in meters
Convert lat_1, lon_1, lat_2, lon_2 to radians;
Δφ ← lat_2 − lat_1;
Δλ ← lon_2 − lon_1;
a ← sin^2(Δφ/2) + cos(lat_1) · cos(lat_2) · sin^2(Δλ/2);
c ← 2 · atan2(√a, √(1 − a));
return R · c;
```

Algorithm 5: GetRecentPositions

Input: Cluster C; current time t_{cur}; time window τ; max count k

Output: List of recent positions from C

```
result ← ∅;
foreach p ∈ C.positions in reverse chronological order do
    if (t_cur − p.time) > τ then
        break;
    Append p to result;
    if |result| ≥ k then
        break;
return result;
```

Complexity: Approximately $O(n \cdot m \cdot k)$; for typical cases $O(n)$.

4.2.4 Handover-Based Cluster Merging

After initial clustering, we merge clusters that can be connected through feasible handovers.

Algorithm 6: Merge Clusters via Handovers

Input: List of clusters C

Output: Merged list of clusters

Constants: $T_H = 28800$ s, $b = 5$, $V_{\max} = 69.4$ m/s

```
Sort C by earliest timestamp;
repeat
    merged ← FALSE;
    for i ← 1 to |C| do
        for j ← i + 1 to |C| do
            if CanHandover(C[i], C[j], T_H, b, V_max) then
                MergeClusters(C[i], C[j]);
                Remove C[j];
                merged ← TRUE;
                break;
        if merged then
            break;
until no merges occur;
return C;
```

Algorithm 7: CanHandover

Input: Clusters C_1, C_2; time window T_H; boundary size b; max speed $V_{\max}$

Output: Boolean indicating feasible handover

```
exit ← last b positions of C_1 by timestamp;
entry ← first b positions of C_2 by timestamp;
foreach p_1 ∈ exit do
    foreach p_2 ∈ entry do
        if |p_2.time − p_1.time| ≤ T_H and IsReachable(p_1, p_2, V_max) then
            return TRUE;
return FALSE;
```

Algorithm 8: MergeClusters

Input: Clusters C_1, C_2

Output: C_1 with all positions from both clusters

```
Append all positions from C_2 to C_1;
Sort C_1.positions by timestamp;
C_1.earliestTime ← min(C_1.earliestTime, C_2.earliestTime);
C_1.latestTime ← max(C_1.latestTime, C_2.latestTime);
```

Complexity: $O(m^3 \cdot b^2)$ with m clusters, b boundary size; m typically small.

4.2.5 Decision Logic

Algorithm 9: MakeDecision

Input: List of clusters C; total position count n

Output: ValidationResult $\in$ {`TRUE`, `FALSE`, `UNCERTAIN`}

Constants: $\alpha = 0.10$, $s_{\min} = 5$

```
if |C| = 0 then
    return UNCERTAIN;
if |C| = 1 then
    return TRUE;
threshold ← max(s_min, α × n);
significant ← {C ∈ C : |C.positions| ≥ threshold};
if |significant| = 0 then
    return UNCERTAIN;
else if |significant| = 1 then
    return TRUE;
else
    return FALSE;
```

4.2.6 Complete Algorithm

Algorithm 10: SingleVehicleCheck: Complete Pipeline

Input: Raw position sequence *rawPositions*

Output: Result with validation status and diagnostics

```
positions ← PreprocessPositions(rawPositions) ;                    // Alg. 1
if |positions| = 0 then
    return Result(UNCERTAIN, "No valid positions");
if |positions| = 1 then
    return Result(TRUE, "Single position");
clusters ← BuildInitialClusters(positions) ;                       // Alg. 2
clusters ← MergeClustersViaHandovers(clusters) ;                   // Alg. 6
result ← MakeDecision(clusters, |positions|) ;                     // Alg. 9
Build diagnostics: total positions, cluster count, sizes, significant count;
return Result(result, diagnostics);
```

4.2.7 Worked Example: Step-by-Step Walkthrough

Figure 4.2 illustrates the SingleVehicleCheck algorithm applied to a concrete dataset of 12 GPS positions from 2 vehicles. We trace each position through the pipeline stages: preprocessing, clustering, component detection, and final decision.

4.2.8 Correctness Analysis

Lemma 4.1 *If all positions belong to a single vehicle, the algorithm returns TRUE.*

Proof. A single vehicle's positions will all be mutually reachable (possibly through intermediate positions) given physical speed constraints. The initial clustering will place them in one component, or handover merging will connect any separated clusters. Thus the final result is a single cluster, and the algorithm returns TRUE.

Lemma 4.2 *If two vehicles are concurrently tracking at distant locations, the algorithm returns FALSE.*

Proof. Positions from different vehicles at distant locations cannot satisfy the reachability predicate (impossible speeds required). They will form separate clusters. Since both have significant positions, they will be counted as significant clusters, and the algorithm returns FALSE.

Theorem 4.3 *The Single Vehicle Check algorithm correctly classifies position sequences.*

Proof. Follows from the two lemmas above, plus the UNCERTAIN case for ambiguous situations (small clusters that could be noise or errors).

4.3 Complexity Analysis

Table 4.1: Complexity summary

Phase	**Time**	**Space**
Preprocessing	$O(n \log n)$	$O(n)$
Initial Clustering	$O(n \cdot m)$	$O(n)$
Handover Merging	$O(m^3 \cdot b^2)$	$O(m)$
Decision	$O(m)$	$O(1)$
Total	$O(n \log n + m^3 \cdot b^2)$	$O(n)$

Where n = number of positions, m = number of clusters (typically $m \ll n$), b = boundary size (constant = 5). For typical data with small m (< 10 clusters), the algorithm runs in $O(n \log n)$ time.

4.4 Error Handling and Edge Cases

4.4.1 Input Validation

The algorithms include robust input validation:

- **Null checks:** Skip positions with null coordinates or timestamps.
- **Range checks:** Validate latitude $[-90, 90]$ and longitude $[-180, 180]$.
- **NaN checks:** Skip positions with NaN values.
- **Empty input:** Return UNCERTAIN for empty position lists.

4.4.2 Edge Cases

Table 4.2: Edge case handling

Edge Case	Handling
No positions	UNCERTAIN
Single position	TRUE
All positions invalid	UNCERTAIN
Zero time difference	Allow 100 m tolerance

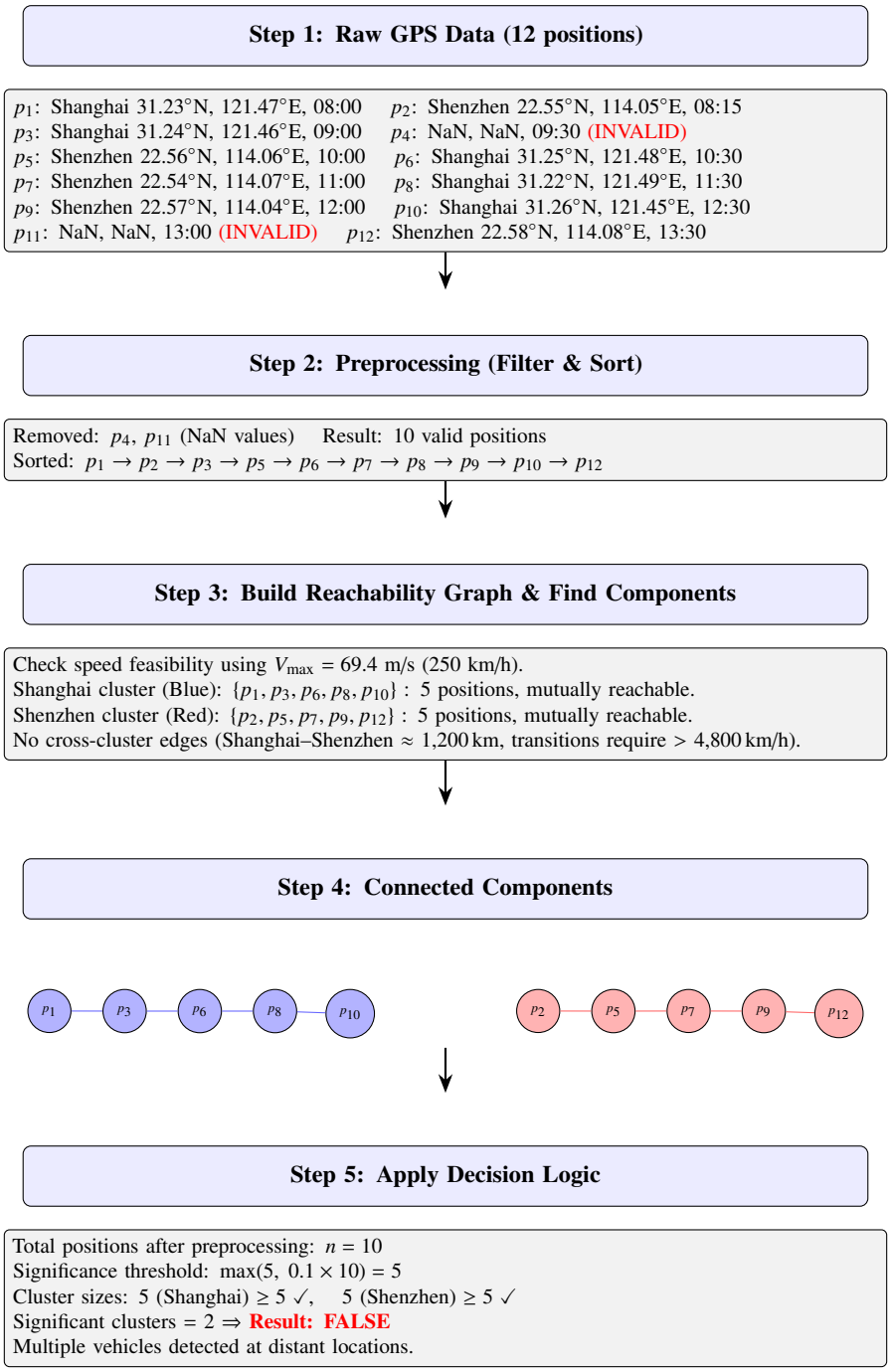


Figure 4.2: Step-by-step walkthrough of the SingleVehicleCheck algorithm on concrete GPS data from Shanghai and Shenzhen. Both clusters meet the significance threshold, confirming multiple vehicles. The algorithm returns FALSE.

4.5 Algorithm Decision Flow

Figure 4.3 presents the complete decision flowchart of the SingleVehicleCheck algorithm, combining all phases from preprocessing through to the final validation result.

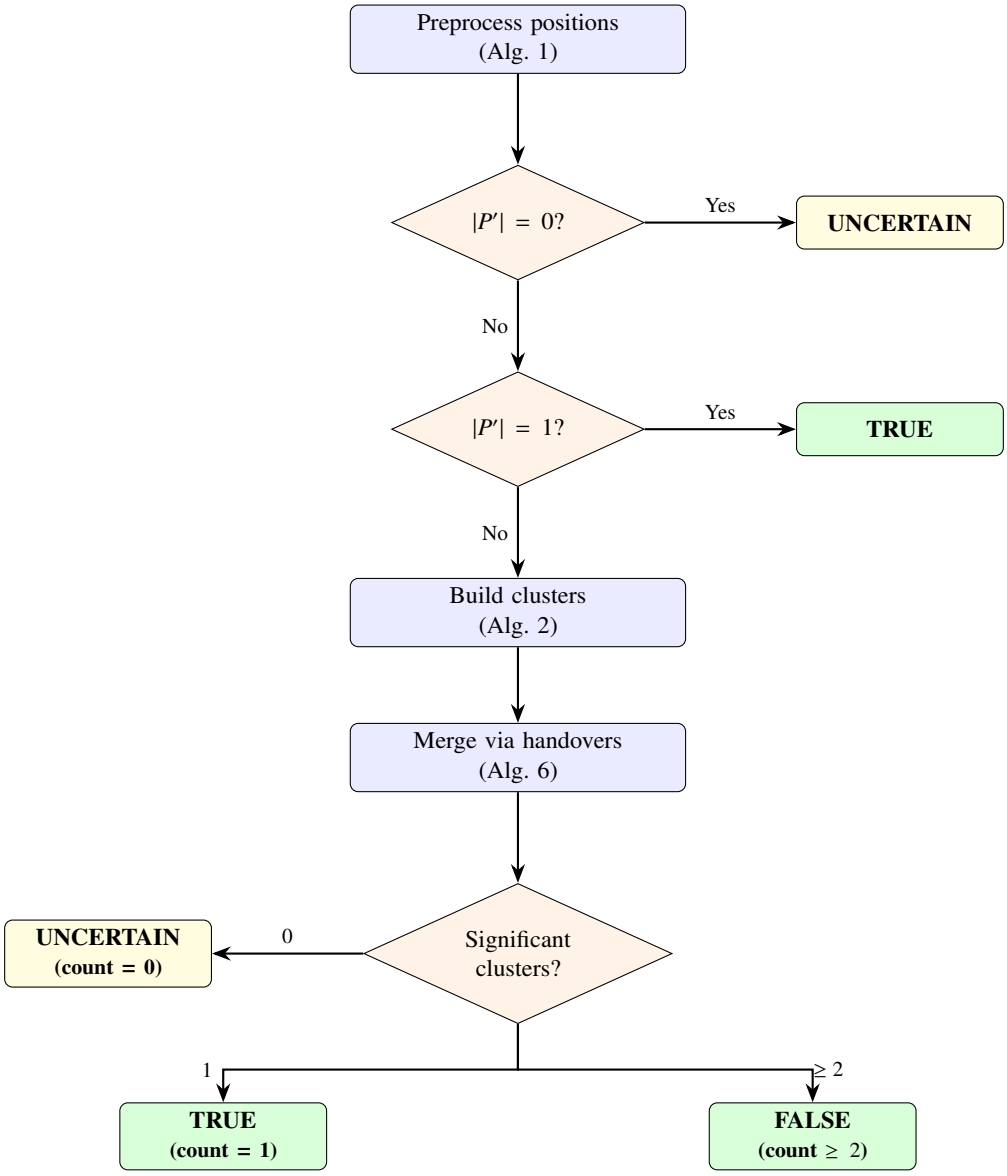


Figure 4.3: Complete decision flow of the SingleVehicleCheck algorithm

4.6 Summary

This chapter presented the complete algorithmic design for single-vehicle validation. The three-phase pipeline (preprocessing, clustering with handover merging, and significance-based decision) transforms raw GPS position data into a deterministic validation verdict. Each phase is specified in pseudocode with explicit correctness arguments and complexity bounds. The overall algorithm runs in $O(n \log n + m^3 \cdot b^2)$ time, which reduces to $O(n \log n)$ for the typical case where the number of clusters m is small. The worked example demonstrated end-to-end execution on a concrete multi-vehicle dataset, confirming that the algorithm correctly identifies concurrent tracking by spatially separated devices.

REFERENCES

[1] DataIntelo Market Research, "Real-time transportation visibility platform market report," DataIntelo, Tech. Rep., 2024.

[2] project44, "project44 named a leader in the 2025 Gartner magic quadrant for real-time transportation visibility platforms," project44 / Gartner, Tech. Rep., 2025.

[3] FourKites, "FourKites announces breakthrough generative AI solution to help companies respond to supply chain disruptions," FourKites, Tech. Rep., 2024.

[4] R. Y. Wang and D. M. Strong, "Beyond accuracy: What data quality means to data consumers," *Journal of Management Information Systems*, vol. 12, no. 4, pp. 5–33, 1996.

[5] International Organization for Standardization, "ISO 19157:2013 – geographic information – data quality," Geneva, Switzerland, 2013.

[6] F. J. Ariza-López, J. Rodríguez-Avi, M. V. Alba-Fernández, and J. L. García-Balboa, "Geospatial data quality (ISO 19157-1): Evolve or perish," *Revista Cartográfica*, 2021.

[7] P. Kuraś *et al.*, "Assessment of the position accuracy of low-cost GNSS sensors," *Sustainability*, vol. 16, 2024.

[8] M. Peretic *et al.*, "Statistical analysis of GNSS multipath errors in urban canyons," in *IEEE/ION Position, Location and Navigation Symposium (PLANS)*. IEEE, 2025.

[9] H. Hubbard and S. Martin, "Performance analysis of a novel hot start vector tracking receiver," in *ION GNSS Conference*, 2010.

[10] A. Oloyede *et al.*, "Technical reliability of GPS/GSM trackers in nigerian urban vs rural fleets," *Preprint / Field Study*, 2024.

[11] T. MacWright, "More than you ever wanted to know about GeoJSON," *Geospatial Data Architecture*, 2015, technical review.

[12] W. Xia, H. Jiang, D. Feng, L. Tian, M. Fu, and Y. Zhou, "A comprehensive study of the past, present, and future of data deduplication," *Proceedings of the IEEE*, vol. 104, no. 9, pp. 1681–1710, 2016.

[13] Gartner Research, "How to improve your data quality," Gartner, Inc., Tech. Rep., 2021.

[14] National Motor Freight Traffic Association, "Secure device provisioning best practices," NMFTA, Tech. Rep., 2023.

[15] 1oT, "Common connectivity failures in fleet deployments," 1oT, Tech. Rep., 2024, industry Whitepaper.

[16] Samsara, "Best practices for managing device health," Samsara Knowledge Base, 2023.

[17] project44, "Why data quality differentiates RTTV platforms," project44 Insights, Tech. Rep., 2021.

[18] ecosio, "Common EDI errors and how to fix them," ecosio B2B Integration Resources, 2025.

[19] DataDocks, "EDI vs API: The hybrid migration for logistics," DataDocks Supply Chain Insights, Tech. Rep., 2024.

[20] W. Yuan *et al.*, "Trajectory outlier detection algorithm for ship AIS data based on dynamic differential threshold," *ResearchGate / AIS Technical Review*, 2020.

[21] J. Chen *et al.*, "UniTraj: A universal foundation model for trajectory modeling," *arXiv preprint*, 2024.

[22] X. Li *et al.*, "Temporal outlier detection in vehicle trajectories," in *IEEE International Conference on Data Engineering (ICDE)*. IEEE, 2009.

[23] S. Zahedi *et al.*, "Estimating activity patterns using spatio-temporal data of cell phone networks," *ResearchGate / Urban Analytics*, 2017.

[24] G. Garcia *et al.*, "Detecting heavy trucks from mobile phone trajectories using CNNs," *Scientific Reports*, 2025.

[25] X. Wang *et al.*, "Unsupervised coordinate inference for e-commerce logistics," *IEEE Transactions on Knowledge and Data Engineering*, 2024.

[26] L. Dai *et al.*, "RealTE: Real-time trajectory estimation over large-scale networks," *Journal of Web Engineering*, 2022.

[27] L. Gong *et al.*, "T-DBSCAN: A spatiotemporal density clustering for GPS trajectory segmentation," *International Journal of Online Engineering*, 2014.

[28] P. Lamichhane, "Graph enhanced trajectory anomaly detection (GETAD)," *arXiv preprint*, 2024.

[29] S. Dabiri and K. Heaslip, "Inferring transportation modes from GPS trajectories using a convolutional neural network," *Transportation Research Part C: Emerging Technologies*, vol. 86, pp. 360–371, 2018.

[30] T. Erdelić *et al.*, "A GPS trajectory segmentation method for transportation mode inference," *Sensors*, 2020.

[31] American Transportation Research Institute, "Cost of truck driver detention to the trucking industry," ATRI, Tech. Rep., 2024.

[32] Descartes MacroPoint, "Demurrage and detention: Understanding the financial impact," Descartes Supply Chain Insights, Tech. Rep., 2024.

[33] Intermodal Logistics, "How to calculate the true impact of a costly delay beyond just the shipping fee," Intermodal Supply Chain Reports, Tech. Rep., 2024.

[34] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. MIT Press, 2022.

[35] T. Vincenty, "Direct and inverse solutions of geodesics on the ellipsoid with application of nested equations," *Survey Review*, vol. 23, no. 176, pp. 88–93, 1975.

[36] European Parliament and Council of the European Union, "Directive 2002/85/EC on the installation and use of speed limitation devices," Official Journal of the European Union, 2002.